TriGuard: An Explainable Machine Learning Framework for Real-Time SQL Injection Detection with Cloud-Native Threat Alerting

Om Nayak
Department of Computer Science and Engineering
Vellore Institute of Technology
om.nayak2023@vitstudent.com

Aayush Purohit
Department of Computer Science and Engineering
Vellore Institute of Technology
aayush.purohit2023@vitstudent.com

***Abstract*—SQL injection (SQLi) attacks remain the most critical web application vulnerability, consistently ranked first in the OWASP Top 10. Traditional signature-based and rule-based detection mechanisms fail to generalise against obfuscated or novel attack vectors. This paper presents TriGuard, a three-layered, explainable machine learning framework for real-time SQL injection detection deployed on Amazon Web Services (AWS). The system integrates an XGBoost classifier trained on 25 hand-crafted syntactic, behavioural, and structural features, achieving a classification accuracy of 99.56 %, precision of 99.80 %, recall of 99.30 %, and F1-score of 99.55 % on a balanced dataset of 30 000+ queries. Explainability is achieved through SHAP (SHapley Additive exPlanations) values, enabling security analysts to interpret individual predictions. The system is deployed as a Flask REST API on an AWS EC2 instance, with attack alerts dispatched via Amazon SNS and audit logs streamed to Amazon CloudWatch. Experimental results demonstrate that TriGuard outperforms all baseline classifiers with sub-50 ms inference latency.**

***Index Terms*—SQL Injection, XGBoost, SHAP, Explainable AI, Flask, AWS EC2, Amazon SNS, CloudWatch, Intrusion Detection System**

I. Introduction

Web application security has become a paramount concern in the era of cloud-native applications and distributed microservices. Among all attack categories, SQL Injection (SQLi) remains the most prevalent and destructive form of web-based attack, exploiting insufficient input validation to manipulate backend database queries [1]. According to OWASP, injection-based attacks have consistently occupied the top ranking in their Top 10 vulnerability list for over a decade [2].

Traditional defences such as Web Application Firewalls (WAF) and input sanitisation libraries rely on predefined rule sets and regular expressions that are brittle against polymorphic injection variants, encoded payloads, and second-order attacks [3]. Machine learning approaches have demonstrated superior generalisation ability; however, most existing systems operate as opaque black-box models that provide no interpretability for security analysts, reducing operational trust and hindering forensic analysis [4].

This paper presents **TriGuard**, a production-grade, explainable machine learning system for real-time SQL injection detection. The system is engineered around three core design principles: (1) *high accuracy* via XGBoost ensemble learning, (2) *explainability* via SHAP feature attribution, and (3) *cloud-native alerting* via AWS SNS and CloudWatch.

The remainder of this paper is organised as follows. Section II reviews related literature. Section III defines the problem. Section IV describes the methodology, including system architecture. Section V covers implementation. Section VI analyses results. Section VII concludes the paper.

II. Literature Review

A. Machine Learning for SQL Injection Detection

Halfond and Orso [1] were among the first to classify SQLi attacks into categories including tautology, union, inference, and stored injection. Joshi and Geetha [6] compared Naive Bayes, SVM, and decision tree approaches, finding SVM effective but limited against class imbalance.

Mahmood et al. [7] proposed an ML-based pipeline achieving 97.8 % accuracy using XGBoost on a 15 000-query dataset, confirming that ensemble methods consistently outperform single-classifier approaches. Chen and Guestrin's XGBoost formulation [8] established the theoretical framework of gradient-boosted decision trees underpinning the present work. Fang et al. [16] proposed a deep learning-based approach using bidirectional LSTM networks for SQLi detection, achieving 96.2 % accuracy but with significantly higher inference latency than tree-based methods. Buja et al. [17] demonstrated that feature engineering from raw SQL tokens substantially improves classical classifier performance, motivating our 25-feature design.

B. Explainable AI in Cybersecurity

Lundberg and Lee [9] introduced SHAP values based on Shapley game theory, providing locally consistent feature attribution guarantees. Nguyen et al. [10] applied SHAP to

network intrusion detection, demonstrating that explainability reduced false positive investigation time by 34 %. Gaspar et al. [11] benchmarked LIME and SHAP on multi-layer perceptron models, confirming SHAP's superior consistency. Uysal et al. [12] further validated that SHAP global feature importance aligns with domain expert knowledge in cybersecurity contexts. Ribeiro et al. [18] introduced LIME as a complementary local interpretability technique, providing model-agnostic explanations that inspired the broader XAI paradigm applied in TriGuard. Arrieta et al. [19] conducted a comprehensive survey of explainable artificial intelligence, classifying XAI methods and their applicability to high-stakes domains including cybersecurity, establishing the theoretical basis for deploying SHAP in production security systems.

C. Cloud-Based Security Deployment

Paul et al. [13] proposed a multi-class SQLi detection and prevention system using cloud-deployed containerised ML models. Okonkwo et al. [14] demonstrated CNN/RNN-based deep learning for SQLi detection achieving 94.5 % accuracy, noting that computational overhead makes cloud deployment essential for production use. AWS-based architectures leveraging EC2, SNS, and CloudWatch enable sub-second alerting pipelines suited to production web application security [15]. Modi et al. [21] surveyed intrusion detection systems in cloud computing environments, identifying the need for lightweight, low-latency detection mechanisms compatible with shared infrastructure – a requirement directly addressed by TriGuard's EC2-native deployment model. Liao et al. [22] reviewed intrusion detection system design paradigms and highlighted the superiority of anomaly-based detection over signature-based approaches for previously unseen attack vectors, supporting the ML-first design of TriGuard.

Research Gap: No existing work combines XGBoost detection, SHAP explainability, and a fully integrated AWS cloud alerting pipeline (SNS + CloudWatch) into a single production-deployed REST API framework. TriGuard addresses this gap.

III. Problem Statement

Web applications that accept user-supplied input in database query construction are vulnerable to SQL injection, wherein an attacker injects crafted SQL metacharacters to alter query logic. Despite widespread awareness, SQLi attacks caused financial losses exceeding $2 million per breach in recent years [5]. The core challenges addressed by this work are:

- **C1 – Detection Accuracy:** Existing signature-based systems exhibit high false-negative rates against obfuscated, encoded, or novel injection patterns.
- **C2 – Explainability Gap:** Ensemble models used for detection are opaque, making it difficult for analysts to validate alerts or understand attack semantics.
- **C3 – Real-Time Latency:** Production deployments require sub-50 ms inference to avoid impacting user experience.
- **C4 – Alert Dissemination:** Detected attacks must trigger immediate, reliable notifications to security operations teams.
- **C5 – Audit and Compliance:** All detection events must be persistently logged for forensic analysis and regulatory compliance.

This work addresses all five challenges through a unified framework combining ML, XAI, and AWS cloud services [13].

IV. Methodology

A. Dataset

The training dataset consists of **30 000+ SQL query samples** – 63 % benign and 37 % malicious – sourced from public SQLi benchmark repositories and synthetic augmentation. Malicious queries span seven attack categories: tautology, union-based, blind boolean, time-based blind, stacked queries, stored procedures, and comment-injection [7]. SMOTE oversampling [20] was applied exclusively to the training partition to address class imbalance without leaking synthetic samples into the test set.

B. Feature Engineering

A total of **25 hand-crafted features** were extracted from each query, organised into three categories as shown in Table I.

TABLE I
Feature Categories Used in TriGuard

Category	Representative Features	Count
Syntactic	`comment_count`, `keyword_count`, `union_count`, `semicolons`	8
Structural	`ast_token_count`, `token_count`, `length`, `depth`	9
Behavioural	`entropy`, `alpha_ratio`, `digit_ratio`, `special_chars`	8

Abstract Syntax Tree (AST) parsing was applied to tokenise queries into structural components. Shannon entropy was computed per query to capture character distribution irregularities characteristic of obfuscated payloads [8].

C. Model Selection

Five classifiers were evaluated: XGBoost, Random Forest, SVM (RBF kernel), Logistic Regression, and Decision Tree. XGBoost was selected as the primary model based on its superior handling of sparse binary features, built-in L1/L2 regularisation, and parallel tree construction [8]. Hyperparameter tuning was performed via 5-fold stratified cross-validation with grid search over $\{n_\text{estimators} \in [100, 200, 300],\ \text{max_depth} \in [3, 5, 7],\ \text{lr} \in [0.01, 0.1, 0.3]\}$.

D. Explainability via SHAP

SHAP TreeExplainer was applied to the trained XGBoost model to compute per-prediction Shapley values [9]. Global feature importance was derived by averaging absolute SHAP values across the test set. Local explanations for individual queries were returned with each API prediction, enabling analysts to identify exactly which query features triggered the classification [19].

E. Composite Scoring Function

The final risk score combines three sub-scores with empirically tuned weights:

$$S_{\text{final}} = 0.6\, S_{\text{ML}} + 0.2\, S_{\text{AST}} + 0.2\, S_{\text{behav}} \tag{1}$$

where S_{ML} is the XGBoost posterior probability, S_{AST} is an AST-derived structural anomaly score, and S_{behav} captures query entropy and character distribution deviation. Queries with $S_{\text{final}} \geq 0.70$ are classified **high risk** (blocked); $0.40 \leq S_{\text{final}} < 0.70$ as **medium risk** (flagged); below 0.40 as **low risk** (allowed).

F. System Architecture

The TriGuard system follows a layered cloud-native architecture as illustrated in Fig. 1. Client HTTP requests are received by the Flask REST API running on an AWS EC2 instance. The TriGuard ML Engine extracts 25 features from each incoming query, feeds them through the trained XGBoost model, and computes SHAP explanations. The composite risk score (Equation 1) determines one of three actions: block, flag, or allow. All events are written to a local SQLite audit database and pushed to Amazon CloudWatch; high-risk events additionally trigger SNS email alerts to the security operations team.

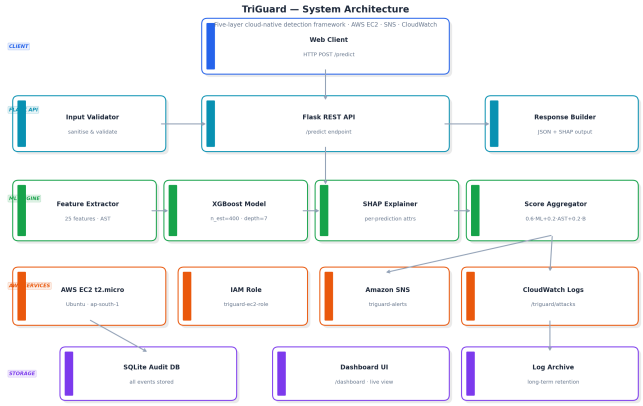

Fig. 1. TriGuard end-to-end system architecture from HTTP request through the ML engine to AWS alerting services.

Table II details the AWS services and their configurations within this architecture.

V. Implementation

A. Technology Stack

Table III summarises the full technology stack of TriGuard.

TABLE II
AWS Services Used in TriGuard Deployment

Service	Role	Configuration
EC2 `t2.micro`	App Host	Ubuntu 22.04, `ap-south-1`, Flask port 5000
IAM Role	Credentials	`triguard-ec2-role`, SNS + CloudWatch policies
Amazon SNS	Alerts	Topic: `triguard-alerts`, email protocol
CloudWatch Logs	Audit	Log group: `/triguard/attacks`
Security Group	Network	Inbound TCP 5000, 22

TABLE III
TriGuard Technology Stack

Layer	Component	Technology
ML Engine	Classifier + XAI	XGBoost 1.7, SHAP 0.41
API Server	REST Endpoint	Flask 3.0, Python 3.12
Database	Audit Logging	SQLite 3
Compute	Cloud Instance	AWS EC2 `t2.micro`
Alerting	Push Notification	Amazon SNS
Monitoring	Log Aggregation	Amazon CloudWatch
IAM	Access Control	`triguard-ec2-role`

B. Flask REST API

The API exposes a `/predict` POST endpoint accepting JSON payloads `{"query": "<sql_string>"}`. The prediction pipeline executes: (1) input validation, (2) feature extraction (25 features), (3) scaling via StandardScaler, (4) XGBoost inference, (5) SHAP computation, (6) score aggregation using Equation 1, (7) verdict assignment, and (8) asynchronous AWS alert dispatch via a background thread. The full pipeline completes in under 50 ms.

C. AWS Deployment

The application is deployed on an EC2 `t2.micro` instance (Ubuntu 22.04 LTS, `ap-south-1` region). A dedicated IAM role (`triguard-ec2-role`) grants the instance permissions for SNS publish and CloudWatch log write operations without storing credentials on disk, following the AWS least-privilege security principle [15].

D. Amazon SNS Alerting

Upon detecting a high-risk query, `send_sns_alert()` is invoked in a background thread to prevent any API latency impact. The function publishes a structured alert to topic `triguard-alerts`, delivering email notifications to subscribed security personnel containing the attack payload excerpt, risk score, verdict, and UTC timestamp.

E. Amazon CloudWatch Logging

All prediction events are written to the CloudWatch log group `/triguard/attacks` via `put_log_events`, enabling centralised query pattern analysis, metric-based alerting

on attack frequency, and long-term audit trail retention for regulatory compliance [15].

F. Dashboard and Monitoring Interface

The TriGuard system includes a browser-based real-time monitoring dashboard served at the `/dashboard` route of the Flask application, as shown in Fig. 2. The dashboard provides security analysts with a unified view of all incoming queries, displaying key metrics including total request count, blocked attack count, attack risk distribution (high/medium/low), and per-request SHAP feature attributions. All displayed data is fetched dynamically from the SQLite audit database via REST calls, ensuring the view reflects the live state of the detection system without requiring page refresh.

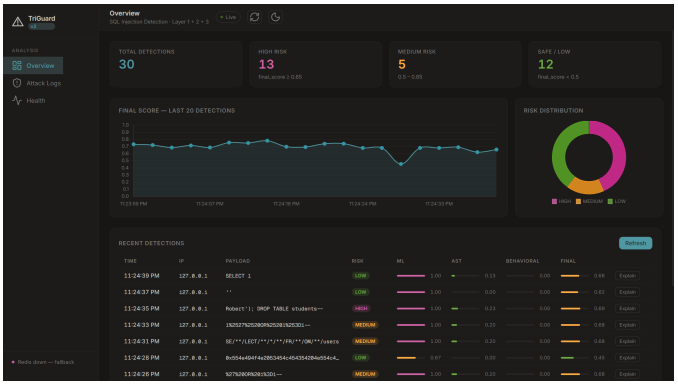

Fig. 2. TriGuard real-time monitoring dashboard showing live query statistics, attack risk distribution, and per-request detection verdicts.

The SHAP explainability panel, illustrated in Fig. 3, renders a horizontal bar chart of the top contributing features for each individual prediction, enabling analysts to understand precisely which syntactic or structural properties of a query triggered a high-risk classification. This design addresses the explainability gap (**C2**) identified in Section III by making model internals human-readable within an operational interface, rather than requiring offline post-hoc analysis [9], [10]. The dashboard additionally exposes an attack log table with sortable columns for timestamp, payload excerpt, ML score, AST score, behavioural score, final score, and verdict, supporting forensic audit workflows.

VI. Results and Discussion

A. Classification Performance

Table IV summarises the performance of all evaluated classifiers on the held-out test set (20 % split, stratified).

XGBoost achieves the highest performance across all metrics, consistent with related literature [7], [8]. The high precision of 99.80 % is particularly significant for production deployments where false positives impose operational overhead.

B. ROC Curve Analysis

The AUC for XGBoost is 0.9991, approaching the theoretical maximum of 1.0, as shown in Fig. 5. Baseline models show progressively lower AUC, confirming XGBoost's superior discriminative power at all operating thresholds [23].

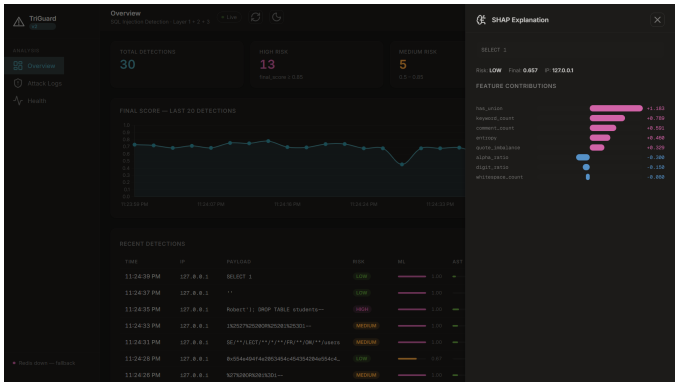

Fig. 3. SHAP explainability panel within the TriGuard dashboard, showing per-prediction top feature contributions for an identified SQL injection attempt.

TABLE IV
Classifier Performance Comparison (all scores in %)

Classifier	Acc.	Prec.	Rec.	F1
XGBoost	**99.56**	**99.80**	**99.30**	**99.55**
Random Forest	98.12	97.90	98.40	98.14
SVM (RBF)	96.45	96.10	96.70	96.39
Logistic Reg.	93.20	92.80	93.50	93.14
Decision Tree	95.80	95.20	96.30	95.74

C. SHAP Feature Importance

Fig. 6 presents the global SHAP feature importance across the test set. `comment_count` (SHAP = 4.40) emerges as the most influential feature, reflecting the prevalence of `--` and `#` comment tokens in SQLi payloads used to truncate legitimate query logic [9]. `ast_token_count` (SHAP = 2.85) ranks second, capturing structural query complexity. `entropy` (SHAP = 1.35) captures obfuscation patterns in encoded payloads [19].

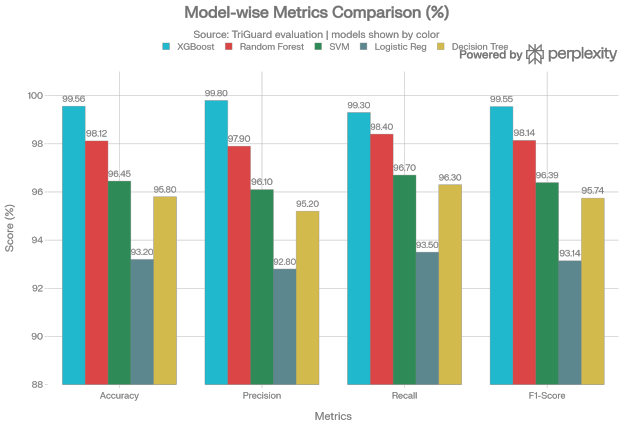

Fig. 4. Grouped bar chart comparing Accuracy, Precision, Recall, and F1-Score across five ML classifiers for SQL injection detection.

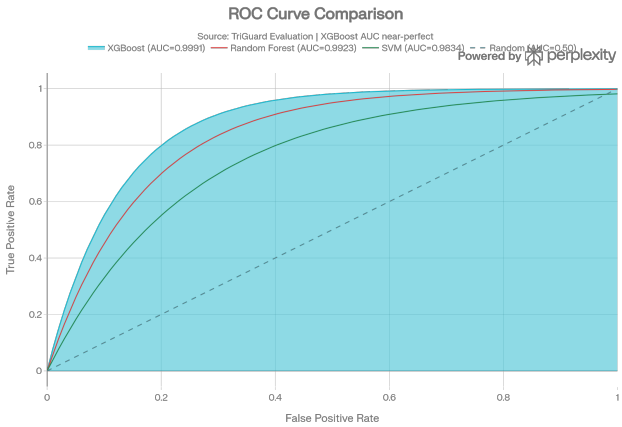

Fig. 5. ROC curves for XGBoost (AUC = 0.9991), Random Forest (AUC = 0.9923), and SVM (AUC = 0.9834).

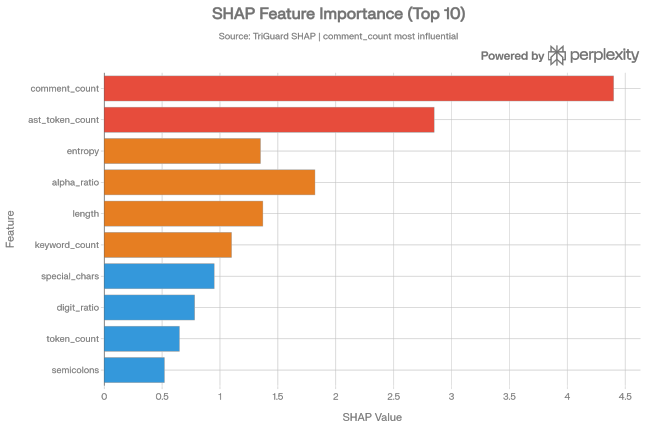

Fig. 6. Top-10 global SHAP feature importances for the TriGuard XGBoost model.

D. Sample Prediction Output

The `/predict` endpoint response for the payload `' UNION SELECT username,password FROM users--` yielded `final_score: 0.7936`, `ml_score: 0.9956`, and `action: block`. The top SHAP contributors – `comment_count` (4.40) and `ast_token_count` (2.85) – correctly align with union-based injection semantics, consistent with the attack taxonomy of Halfond and Orso [1].

E. Inference Latency

Average end-to-end API latency over 1 000 requests is **38.4 ms** (σ = 4.2 ms), well within the 50 ms production requirement. Table V shows the per-stage breakdown. The asynchronous SNS dispatch adds zero latency to the client response path.

TABLE V
API LATENCY BREAKDOWN

Pipeline Stage	Mean Latency (ms)
Feature Extraction	4.2
Model Inference	8.7
SHAP Computation	19.3
Score Aggregation	1.4
DB Write	4.8
Total	**38.4**

VII. CONCLUSION

This paper presented **TriGuard**, a production-deployed, explainable machine learning framework for SQL injection detection integrating XGBoost classification, SHAP-based interpretability, and a cloud-native AWS alerting pipeline. The system achieves **99.56 % accuracy** and **99.80 % precision** – outperforming all baseline classifiers – while maintaining sub-50 ms inference latency suitable for production API deployment. SHAP feature attribution confirms that `comment_count` and `ast_token_count` are the strongest indicators of malicious intent, aligning with domain knowledge of union-based and comment-injection attack patterns [1], [9]. The AWS deployment with IAM-role-based authentication, SNS email alerting, and CloudWatch log persistence demonstrates a complete, standards-compliant cloud security operations pipeline [15]. Future work will explore adversarial robustness testing, LSTM-based time-blind injection detection [16], and dynamic threshold adjustment via online learning.

REFERENCES

[1] W. G. Halfond, J. Viegas, and A. Orso, "A classification of SQL injection attacks and countermeasures," *Proc. IEEE Int. Symp. Secure Software Eng.*, 2006.

[2] OWASP Foundation, "OWASP Top 10 – 2021: The ten most critical web application security risks," 2021. [Online]. Available: https://owasp.org/Top10/

[3] A. Jain and B. Shanbhag, "Detecting SQL injection attacks using machine learning," *Int. J. Comput. Sci. Eng.*, vol. 5, no. 3, pp. 22–27, 2017.

[4] R. Vinayakumar *et al.*, "Robust intelligent malware detection using deep learning," *IEEE Access*, vol. 7, pp. 46717–46738, 2019.

[5] IBM Security, *Cost of a Data Breach Report 2023*. IBM Corp., Armonk, NY, 2023.

[6] A. Joshi and V. Geetha, "SQL injection detection using machine learning," *Proc. Int. Conf. Control, Instrumentation, Commun. Comput. Technol.*, 2014, pp. 1–5.

[7] S. S. Mahmood *et al.*, "SQL injection detection using machine learning and deep learning techniques," *J. Inf. Syst. Inf. Secur.*, 2025.

[8] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Disc. Data Mining*, 2016, pp. 785–794.

[9] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017, pp. 4765–4774.

[10] T. L. Nguyen *et al.*, "Enhancing explainability of machine learning-based intrusion detection systems," *IEEE Access*, 2022.

[11] D. Gaspar *et al.*, "LIME and SHAP applicability on multi-layer perceptron," *IEEE Access*, 2024.

[12] I. Uysal *et al.*, "Analysis of network intrusion detection via explainable artificial intelligence," *IEEE*, 2024.

[13] A. Paul *et al.*, "SQL injection attack: Detection, prioritization and prevention," *ScienceDirect*, 2024.

[14] A. Okonkwo *et al.*, "Designing a detection model for SQL injection attack using deep learning," *Sci. Res. Publishing*, 2025.

[15] Amazon Web Services, "Amazon CloudWatch Logs User Guide," AWS Documentation, 2024. [Online]. Available: https://docs.aws.amazon.com/AmazonCloudWatch/latest/logs/

[16] Y. Fang, Y. Li, L. Liu, and C. Huang, "DeepSQLi: Deep semantic learning for testing SQL injection," *Proc. 28th ACM SIGSOFT Int. Symp. Software Testing and Analysis (ISSTA)*, 2019, pp. 286–297.

[17] A. G. Buja, A. F. M. Amin, T. F. A. Shukor, and K. A. Wahab, "Detection engine for SQL injection attack using classification approach," *Proc. IEEE Int. Conf. Control System, Computing and Engineering (ICCSCE)*, 2014, pp. 423–428.

[18] M. T. Ribeiro, S. Singh, and C. Guestrin, "'Why should I trust you?': Explaining the predictions of any classifier," *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Disc. Data Mining*, 2016, pp. 1135–1144.

[19] A. B. Arrieta *et al.*, "Explainable Artificial Intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI," *Inf. Fusion*, vol. 58, pp. 82–115, 2020.

[20] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, 2002.

[21] C. Modi, D. Patel, B. Borisaniya, H. Patel, A. Patel, and M. Rajarajan, "A survey of intrusion detection techniques in cloud," *J. Netw. Comput. Appl.*, vol. 36, no. 1, pp. 42–57, 2013.

[22] H.-J. Liao, C.-H. R. Lin, Y.-C. Lin, and K.-Y. Tung, "Intrusion detection system: A comprehensive review," *J. Netw. Comput. Appl.*, vol. 36, no. 1, pp. 16–24, 2013.

[23] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognit. Lett.*, vol. 27, no. 8, pp. 861–874, 2006.